

XÂY DỰNG HỆ THỐNG SELF-EVOLVING AI AGENTS TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG

Nguyễn Thanh Hải

optimus1072005@gmail.com

Lĩnh vực Data Science & AI Engineering - Chương trình Viettel Digital Talent 2026

Nguyễn Duy Hưng¹

hungnd1235@gmail.com

Nguyễn Duy Anh¹

nguyenduyanh821998@gmail.com

Bùi Quang Hưng¹

contact.hung.bq@gmail.com

¹Phòng Giải pháp di động - Trung tâm Nghiên cứu Công nghệ & Chuyển đổi số - Tổng công ty Mạng lưới Viettel

1. GIỚI THIỆU CHUNG

Các mạng viễn thông hiện đại, đặc biệt trong bối cảnh 5G, trung tâm dữ liệu và mạng điều khiển bằng phần mềm, có quy mô lớn, nhiều lớp giao thức và phụ thuộc chặt chẽ giữa thiết bị, dịch vụ, cấu hình định tuyến, topology và chỉ số hiệu năng. Một sự cố nhỏ như sai ASN BGP, IP missing, link down, switch BMv2 dừng hoạt động hoặc rule SDN sai có thể gây mất kết nối diện rộng, suy giảm chất lượng dịch vụ và làm tăng chi phí vận hành. Trong thực tế, xử lý sự cố không chỉ là phát hiện bất thường, mà còn cần định vị thiết bị lỗi, xác định nguyên nhân (Root Cause Analysis - RCA) và đưa ra hướng khắc phục có thể kiểm chứng. Bài toán này khó vì triệu chứng quan sát được thường gián tiếp, chịu nhiễu và bắt nguồn từ nhiều tầng cấu hình khác nhau, gây khó khăn lớn cho kỹ sư vận hành.

Các cách tiếp cận truyền thống thường dựa trên luật thủ công, kinh nghiệm chuyên gia hoặc mô hình học máy tĩnh. Những phương pháp này khó thích ứng khi topology thay đổi, khi xuất hiện dạng lỗi mới hoặc khi dữ liệu vận hành có nhiễu. Sự phát triển của mô hình ngôn ngữ lớn (LLM) và AI Agent mở ra khả năng mới: agent có thể đọc mô tả sự cố, sử dụng công cụ chẩn đoán, suy luận nhiều bước và giải thích kết quả bằng ngôn ngữ tự nhiên. Các nghiên cứu gần đây về LLM cho chẩn đoán mạng nhấn mạnh tiềm năng kết hợp tri thức miền, log vận hành và công cụ kiểm chứng thay vì chỉ dựa vào câu trả lời một lượt của mô hình. Tuy nhiên, agent thông thường vẫn mang tính tĩnh; nếu prompt, bộ nhớ hoặc mô tả công cụ không được cập nhật thì hiệu quả chẩn đoán dễ suy giảm khi môi trường mạng thay đổi. Vì vậy, hệ thống cần học từ trace gọi công cụ, phản hồi chấm điểm và các case đã xử lý để điều chỉnh quy trình suy luận cho những sự cố tiếp theo.

Đề tài này nghiên cứu và xây dựng hệ thống **Self-Evolving AI Agents** cho xử lý sự cố mạng viễn thông trên nền tảng benchmark NIKA [1]. Khác với agent tĩnh, self-evolving agent khai thác chính lịch sử xử lý nhiệm vụ, phản hồi và kết quả đánh giá làm nguồn dữ liệu để cập nhật bộ nhớ, công cụ hoặc chiến lược suy luận [2]. Mục tiêu của đề tài là biến mỗi lần chẩn đoán thành một nguồn kinh nghiệm giúp agent cải thiện ở các lần sau. Cụ thể, hệ thống cần tiếp nhận dữ liệu từ môi trường mạng giả lập, gọi các công cụ kiểm tra trạng thái mạng, suy luận nguyên nhân gốc rễ, gửi kết quả đánh giá và cập nhật tri thức chẩn đoán dài hạn.

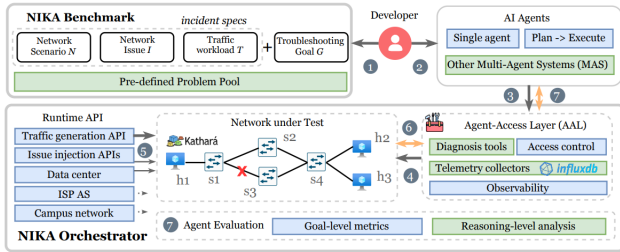
Ý nghĩa thực tiễn của sản phẩm nằm ở việc hỗ trợ kỹ sư vận hành thực hiện nhanh các bước lặp lại như kiểm tra kết nối, đọc cấu hình, đối chiếu bảng định tuyến, phân tích log và tổng hợp bằng chứng. Hệ thống không thay thế chuyên gia, mà đóng vai trò trợ lý chẩn đoán giúp khoanh vùng nguyên nhân, chuẩn hóa báo cáo sự cố và lưu lại kinh nghiệm đã kiểm chứng để tái sử dụng cho các ca tương tự. Đóng góp chính của sinh viên gồm:

- Thiết kế và hiện thực workflow agent chẩn đoán trên nền tảng NIKA/Kathará, tích hợp các dịch vụ MCP để agent tương tác với router, host, switch P4/BMv2, telemetry và hệ thống chấm điểm.
- Đề xuất hai mô-đun tự tiến hóa gồm **Procedural Memory** lấy cảm hứng từ Skill-Pro [3] và **Tool Refinement** lấy cảm hứng từ DRAFT [4], giúp agent lưu trữ kỹ năng chẩn đoán và tinh chỉnh tài liệu công cụ dựa trên trace thực tế.
- Xây dựng pipeline benchmark để so sánh baseline và agent tự tiến hóa trên NIKA, đánh giá theo detection, localization, RCA, số bước, số lần gọi công cụ, tool error rate, token và thời gian chạy.

2. NỘI DUNG VÀ PHƯƠNG PHÁP

2.1. Giới thiệu về benchmark NIKA

NIKA được giới thiệu như một network arena để đánh giá AI Agent trong bài toán xử lý sự cố mạng [1]. Benchmark này giải quyết một khoảng trống quan trọng: trước đây việc đánh giá agent chẩn đoán mạng thường thiếu môi trường chuẩn hóa, khó tái lập và tốn nhiều công sức vận hành. Như minh họa trong Hình 1, NIKA cung cấp các kịch bản mạng có thể replay, giao diện agent-network rõ ràng và tập incident được thiết kế để kiểm tra khả năng phát hiện lỗi, định vị thiết bị lỗi và xác định nguyên nhân gốc rễ. Vì vậy, NIKA phù hợp với mục tiêu của đề tài: xây dựng một agent không chỉ trả lời bằng ngôn ngữ tự nhiên, mà còn phải sử dụng công cụ, thu thập bằng chứng và cải thiện qua nhiều lần xử lý sự cố.



Hình 1. Tổng quan benchmark NIKA cho đánh giá AI Agent trong xử lý sự cố mạng.

2.2. Khung tổng quan hệ thống

Hệ thống được xây dựng quanh một agent chẩn đoán có khả năng tương tác với môi trường mạng thông qua các công cụ MCP. Ở mức tổng quan, môi trường mạng cung cấp topology, thiết bị và dữ liệu quan sát; agent sử dụng các công cụ để kiểm tra trạng thái mạng; bộ chấm điểm đánh giá kết quả cuối cùng; và hai mô-đun tự tiến hóa cập nhật tri thức chẩn đoán sau mỗi episode. Hai mô-đun này gồm **Procedural Memory** để lưu kinh nghiệm dưới dạng quy trình chẩn đoán và **Tool Refinement** để cải thiện tài liệu công cụ dựa trên trace thực thi.

2.2.1. Công cụ và công nghệ sử dụng

Hệ thống sử dụng NIKA làm benchmark chính để tạo các incident mạng và chấm điểm kết quả chẩn đoán. Kathará [5] được dùng để ảo hóa topology bằng container, giúp khởi động lab, tiêm lỗi và tái lập môi trường một cách nhất quán. MCP/FastMCP [6] đóng vai trò lớp giao tiếp giữa agent và môi trường mạng: các thao tác như kiểm tra kết nối, đọc cấu hình, truy vấn telemetry hoặc nộp kết quả đều được chuẩn hóa thành công cụ. LangGraph/LangChain [7] được dùng

để điều phối workflow agent, trong đó các chiến lược chính gồm ReAct [8], Plan & Execute và Reflexion [9].

Ở tầng dữ liệu và mạng, hệ thống dùng InfluxDB để lưu telemetry, FRRouting [10] cho các kịch bản định tuyến và P4/BMv2 [11] cho các kịch bản data-plane lập trình được. Phần triển khai được viết bằng Python 3.12 và quản lý môi trường bằng uv. Các nhóm công cụ MCP chính gồm Kathará Base, FRR, BMv2/P4, Telemetry, Task Management và Tool Evolution; mỗi nhóm tương ứng với một phần quan sát hoặc điều khiển cần thiết trong quá trình agent xử lý sự cố.

2.2.2. Quy trình triển khai

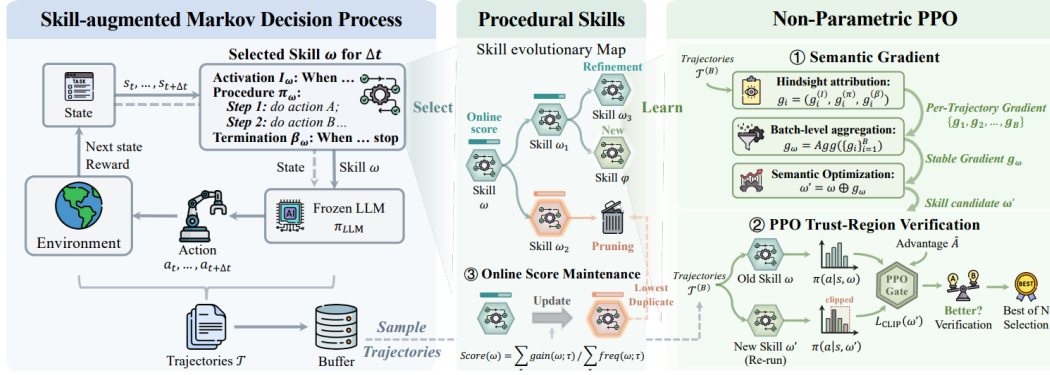
Quy trình triển khai bắt đầu từ việc chọn tập case trong benchmark, chuẩn bị cấu hình lab Kathará và xác định loại lỗi cần tìm tương ứng với từng kịch bản. Mỗi lab được khởi động trong container riêng biệt để đảm bảo môi trường sạch và tái lập được giữa các episode. Sau khi lab sẵn sàng, lỗi được tiêm theo kịch bản định sẵn, agent nhận thông tin khởi đầu về topology và triệu chứng ban đầu rồi bắt đầu vòng chẩn đoán. Toàn bộ trace gồm suy luận, lời gọi công cụ, tham số và quan sát trả về đều được ghi lại để phục vụ chấm điểm và làm dữ liệu đầu vào cho hai mô-đun tự tiến hóa.

Luồng xử lý trong mỗi episode được tách thành hai pha. Trong **Diagnosis Phase**, agent có quyền dùng đầy đủ các công cụ quan sát để thu thập bằng chứng, kiểm tra giả thuyết và loại trừ nguyên nhân không phù hợp. Trong **Submission Phase**, agent chỉ còn công cụ nộp kết quả gồm `is_anomaly`, `faulty_devices` và `root_cause_name`, nhờ đó tránh tình trạng vừa chẩn đoán vừa thay đổi kết luận một cách thiếu kiểm soát. Với baseline, mỗi episode hoàn toàn độc lập và agent không mang kinh nghiệm nào từ trước. Với evolved agent, bộ nhớ kỹ năng và tài liệu công cụ đã được tinh chỉnh từ các episode trước được nạp vào ngữ cảnh trước khi bắt đầu case mới; sau khi episode kết thúc, **Procedural Memory** và **Tool Refinement** tiếp tục được cập nhật từ trace và kết quả chấm điểm.

2.3. Mô-đun Procedural Memory

Procedural Memory chuyển kinh nghiệm chẩn đoán từ dạng lịch sử trajectory dài sang các kỹ năng tái sử dụng [3]. Thay vì đọc lại toàn bộ trajectory cũ, hệ thống rút gọn kinh nghiệm thành các đơn vị kỹ năng có điều kiện kích hoạt, quy trình thực thi và điều kiện dừng rõ ràng. Như Hình 2, trace sau episode được dùng để sinh hoặc tinh chỉnh kỹ năng, giúp định hướng chuỗi hành động cho các episode tiếp theo.

Mỗi kỹ năng gồm ba thành phần: *Activation condition* xác định tình huống kích hoạt (như mất kết nối, lỗi định tuyến hoặc sự cố P4); *Execution procedure* chỉ ra chuỗi kiểm tra ưu tiên (reachability,



Hình 2. Quy trình học kỹ năng tái sử dụng cho mô-đun **Procedural Memory**.

config, routing table); và *Termination condition* xác định thời điểm dừng khi đủ bằng chứng thiết bị lỗi hoặc giả thuyết bị loại bỏ, giúp định hướng trực tiếp quá trình ra quyết định.

Tại mỗi incident, hệ thống truy xuất các kỹ năng phù hợp nhất dựa trên triệu chứng và topology ban đầu. Bộ chọn memory ưu tiên các kỹ năng có hiệu quả lịch sử cao để nạp vào prompt. Để xếp hạng, hệ thống duy trì bảng thống kê tần suất sử dụng, tỷ lệ thành công và chi phí tài nguyên của từng kỹ năng. Kỹ năng chỉ đóng vai trò định hướng hành vi: agent vẫn chủ động thực thi công cụ và có thể dừng kỹ năng nếu quan sát thực tế không còn khớp với điều kiện kích hoạt ban đầu.

Sau mỗi episode, hệ thống dùng phản hồi chấm điểm để cập nhật bộ nhớ. Trajectory thành công dùng để tạo hoặc tinh chỉnh kỹ năng; trajectory thất bại dùng để điều chỉnh điều kiện kích hoạt hoặc các bước gây nhiễu. Cơ chế semantic gradient tạo ra các sửa đổi, được kiểm duyệt qua PPO gate phi tham số [12] để đảm bảo chất lượng. Bộ nhớ cũng duy trì điểm chất lượng theo thời gian để lọc bỏ các kỹ năng trùng lặp hoặc kém hiệu quả. Các kỹ năng dưới ngưỡng chất lượng hoặc ít được gọi sẽ bị loại bỏ để duy trì kích thước bộ nhớ tối ưu.

2.4. Mô-đun Tool Refinement

Tool Refinement xử lý một nguồn lỗi quan trọng khác của agent: tài liệu mô tả công cụ có thể đầy đủ với kỹ sư vận hành nhưng lại thiếu các chi tiết cần thiết để LLM ra quyết định chính xác [4]. Trong các bài toán chẩn đoán mạng phức tạp, việc thực thi công cụ phụ thuộc lớn vào loại thiết bị, tên container, giao thức đang chạy và định dạng đầu ra. Nếu tài liệu thiếu các tiền điều kiện rõ ràng hoặc ví dụ cụ thể, agent dễ gặp lỗi gọi sai thiết bị, sai tham số hoặc hiểu sai kết quả trả về. Do đó, mô-đun này tập trung tinh chỉnh tài liệu mô tả công cụ bằng cách bổ sung chi tiết vận hành thực tế mà không cần sửa đổi mã nguồn gốc của công cụ, giúp tối ưu hóa hiệu năng chẩn đoán.

Bảng 1. Thuật toán học của mô-đun **Tool Refinement**

Input	Tập tài liệu công cụ $\mathcal{D}$, số vòng I , ngưỡng tương đồng ϕ , ngưỡng dừng τ .
Output	Tập tài liệu đã tinh chỉnh $\tilde{\mathcal{D}}$.
1	$\tilde{\mathcal{D}} \leftarrow \emptyset$.
2	for mỗi tài liệu $t \in \mathcal{D}$ do
3	for $i = 1 \dots I$ do
4	Explorer sinh tình huống khám phá e_i .
5	Nếu e_i quá giống lịch sử theo ϕ , sinh lại truy vấn khác.
6	Thực thi công cụ và ghi nhận kết quả r_i .
7	Analyzer rút lỗi tài liệu và gợi ý sửa đổi s_i .
8	Rewriter cập nhật t_i và hướng khám phá tiếp theo.
9	Nếu độ thay đổi $\Delta(t_{i-1}, t_i) > \tau$, dừng vòng học.
10	end for
11	$\tilde{\mathcal{D}} \leftarrow \tilde{\mathcal{D}} \cup \{t_i\}$.
12	end for
13	return $\tilde{\mathcal{D}}$.

Quy trình học được mô tả trong Bảng 1. Explorer đóng vai trò tiên phong tạo các tình huống sử dụng công cụ từ tài liệu hiện tại và lịch sử khám phá. Để đảm bảo độ bao phủ rộng, Explorer áp dụng ràng buộc tương đồng ϕ : nếu một tình huống mới quá giống lịch sử, hệ thống yêu cầu sinh lại truy vấn khác, ghi nhận toàn bộ tham số, kết quả thực thi và lỗi phát sinh làm nguồn dữ liệu huấn luyện phong phú.

Analyzer sau đó đối chiếu kết quả thực thi để phát hiện khoảng cách giữa hành vi thực tế và tài liệu, từ đó sinh các đề xuất chỉnh sửa cụ thể. Rewriter kết hợp các đề xuất này để viết lại tài liệu mới và định hướng Explorer ở vòng tiếp theo cho đến khi tài liệu đạt trạng thái hội tụ τ . Cách thiết kế này đảm bảo cải thiện hiệu quả chẩn đoán nhưng vẫn giữ ranh giới an toàn tuyệt đối cho hệ thống mạng.

3. THỰC NGHIỆM

3.1. Thiết lập thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên hệ thống trang bị CPU 12 nhân và 16 GB RAM. Các mô hình ngôn ngữ lớn được gọi trực tiếp qua Netmind API (gồm GPT-OSS-120B và Qwen3.5-122B-A10B-FP8) thay vì triển khai cục bộ. Cấu hình mặc định để báo cáo kết quả là workflow ReAct kết hợp với GPT-OSS-120B; các workflow Plan & Execute và Reflexion được sử dụng làm đối chiếu để đánh giá ảnh hưởng của phương pháp tổ chức suy luận.

Thực nghiệm được tiến hành trên hai cấu hình: **Baseline** và **Evolved**. Trong đó, cấu hình **Evolved** tích hợp mô-đun **Procedural Memory** với tham số `top_k=5`, bộ nhớ giới hạn 1500 tokens, `skill_max_age=4`, `skill_pool=32`, bộ chọn LCB (`min_lcb=-0.05`), bộ điều khiển heuristic (`PP0_epsilon=0.05`), `nominee_k=3` và số mẫu tiến hóa bằng 3. Mô-đun **Tool Refinement** được thiết lập với `num_docs=6`, `num_scoped_docs=4`, chiều dài tài liệu `doc_chars=500`, tần suất kiểm tra `planned_check=4`, `next_checks=2` và ngưỡng hội tụ bằng 0.75 cho từng tiến trình chẩn đoán.

Bảng 2. Tóm tắt phạm vi benchmark trong thực nghiệm

Nhóm sự cố	Số lượng
Link Failures	12
End-Host Failures	16
Misconfigurations	27
Resource Contention	13
Network Node Errors	20
Network Under Attack	12
No-fault	25
Tổng	125

Tập benchmark con gồm 125 case được thiết kế theo dạng curriculum-evaluation: 100 case lỗi (gồm 44 biến thể tiến hóa và 56 case đại diện cho từng nguyên nhân gốc rễ) được sắp xếp có chủ đích để đánh giá sự tiến bộ của agent; 25 case no-fault được xen kẽ tất định để kiểm chứng khả năng phân biệt mạng bình thường.

3.2. Kết quả thực nghiệm

Bảng 3 tổng hợp kết quả chạy ReAct trên cả hai mô hình ở chế độ Baseline và Evolved, bao gồm toàn bộ metric chẩn đoán và chi phí vận hành trên 125 case.

Hình 3 theo dõi diễn biến metric qua từng batch 25 case để quan sát xu hướng cải thiện của Evolved Agent theo thời gian. Bảng 4 so sánh ba workflow ReAct, Plan&Execute và Reflexion trên cùng cấu hình Evolved để tách biệt ảnh hưởng của cách tổ chức suy luận.

Placeholder figure: curve Detection, Localization và RCA theo từng batch 25 case. Cấu hình: ReAct Evolved trên GPT-OSS-120B và Qwen3.5-122B-A10B-FP8.

Hình 3. Diễn biến metric theo từng batch 25 case của ReAct Evolved trên hai mô hình nền.

Placeholder figure: độ ổn định qua 3 lần chạy. Cấu hình: ReAct trên Baseline và Evolved với GPT-OSS-120B.

Hình 4. So sánh độ ổn định qua ba lần chạy của ReAct Baseline và ReAct Evolved trên GPT-OSS-120B.

Hình 4 đánh giá độ ổn định bằng cách lặp lại ba lần chạy độc lập trên cùng tập case với cấu hình ReAct + GPT-OSS-120B, nhằm kiểm tra mức độ nhất quán của kết quả giữa Baseline và Evolved.

4. ĐÁNH GIÁ HIỆU QUẢ

4.1. Tiêu chí đánh giá

Basic. Đo chất lượng chẩn đoán qua detection, localization, RCA và incident success rate, kết hợp với chi phí vận hành gồm số bước suy luận, số lần gọi công cụ, lỗi gọi công cụ, token đầu vào/đầu ra và runtime.

Evolve. Đánh giá liệu **Procedural Memory** và **Tool Refinement** có giúp agent cải thiện qua các episode hay không, qua các tín hiệu như kỹ năng được truy xuất và tái sử dụng, mức giảm lỗi gọi công cụ và mức độ hội tụ của tài liệu công cụ sau nhiều vòng cập nhật.

4.2. Phân tích chuyên sâu

4.3. Hạn chế và rủi ro

4.4. Hướng đi tương lai

Bảng 3. Kết quả chung của ReAct trên Baseline và Evolved với GPT-OSS-120B và Qwen3.5-122B-A10B-FP8

Metric	GPT-OSS Baseline	GPT-OSS Evolved	Qwen Baseline	Qwen Evolved
Detection	0.840	0.872 (+3.8%)	0.653	0.780 (+19.4%)
Localization Acc.	0.368	0.384 (+4.3%)	0.388	0.508 (+30.9%)
Localization F1	0.411	0.459 (+11.7%)	0.466	0.599 (+28.5%)
RCA Acc.	0.304	0.376 (+23.7%)	0.327	0.525 (+60.6%)
RCA F1	0.304	0.387 (+27.3%)	0.340	0.537 (+57.9%)
Incident Success	0.208	0.280 (+34.6%)	0.367	0.407 (+10.9%)
Avg. Steps	15.29	12.74 (-16.7%)	27.14	24.17 (-11.0%)
Avg. Tool Calls	13.60	11.09 (-18.5%)	42.10	40.20 (-4.5%)
Tool Error Rate	0.0041	0.0000 (-100%)	0.0004	0.0000 (-100%)
Input Tokens	127.5k	143.9k (+12.9%)	586.3k	538.8k (-8.1%)
Output Tokens	5.6k	5.7k (+1.8%)	5.9k	6.1k (+3.4%)
Total Tokens	133.1k	149.6k (+12.4%)	592.2k	544.9k (-8.0%)
Runtime (s)	99.68	106.83 (+7.2%)	205.20	204.94 (-0.1%)

Bảng 4. Ablation study trên Evolved Agent khi thay đổi workflow

Metric	ReAct	Plan&Execute	Reflexion
Detection	0.816	–	–
Localization	0.399	–	–
RCA	0.369	–	–
Input Token	263.8k	–	–
Output Token	5.8k	–	–
Runtime (s)	133.96	–	–

5. KẾT LUẬN

Đề tài đã xây dựng được một prototype Self-Evolving AI Agents cho xử lý sự cố mạng viễn thông trên nền tảng NIKA. Hệ thống tích hợp môi trường mạng giả lập, các MCP server chẩn đoán, workflow agent và hai mô-đun tự tiến hóa gồm **Procedural Memory** và **Tool Refinement**. Kết quả thực nghiệm cho thấy hướng tiếp cận này giúp agent xử lý sự cố có cấu trúc hơn, cải thiện khả năng xác định nguyên nhân gốc rễ và giảm các thao tác công cụ kém hiệu quả mà không cần huấn luyện lại mô hình nền. Tính mới của đề tài nằm ở việc xem agent như một hệ thống có thể học sau mỗi episode thay vì một thành phần tĩnh chỉ phụ thuộc vào prompt ban đầu. **Procedural Memory** giúp lưu lại quy trình chẩn đoán có điều kiện kích hoạt, còn **Tool Refinement** giúp cải thiện cách agent hiểu và sử dụng công cụ dựa trên trace thực tế. Hai mô-đun này tạo thành một vòng lặp học kinh nghiệm phù hợp với bài toán vận hành mạng, nơi mỗi sự cố đã xử lý có thể trở thành nguồn tri thức cho các sự cố tương tự về sau. Về đóng góp cá nhân, sinh viên đã trực tiếp nghiên cứu các hướng self-evolving agent, thiết kế kiến trúc tích hợp

với NIKA, hiện thực hai mô-đun học kinh nghiệm, xây dựng pipeline benchmark và phân tích kết quả. Trong các bước phát triển tiếp theo, hệ thống cần được tối ưu khả năng chọn lọc memory, giảm chi phí token, tăng khả năng tổng quát hóa sang topology/giao thức mới và bổ sung cơ chế human-in-the-loop để kiểm soát tri thức được lưu lâu dài. Khi các điểm này được hoàn thiện, hệ thống có thể phát triển thành trợ lý chẩn đoán hỗ trợ kỹ sư vận hành mạng trong việc thu thập bằng chứng, khoanh vùng nguyên nhân và chuẩn hóa báo cáo sự cố.

TÀI LIỆU THAM KHẢO

- [1] Zhihao Wang, Alessandro Cornacchia, Alessio Sacco, Franco Galante, Marco Canini, and Dingde Jiang. A network arena for benchmarking AI agents on network troubleshooting. *arXiv preprint arXiv:2512.16381*, 2025. URL <https://arxiv.org/abs/2512.16381>. 1, 2
- [2] Huan-ang Gao, Jiayi Geng, Wenyue Hua, et al. A survey of self-evolving agents. *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2507.21046>. 1
- [3] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills via non-parametric PPO for LLM agents. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.01869>. 1, 2
- [4] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. DRAFT: Tool mastery via self-driven interactions. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.08197>. 1, 3
- [5] Stefano Marrone, Luca Nicoletti, and Claudio Pizzuti. Kathará: a container-based framework for network emulation. In *IEEE International Conference on Computer Communication and Networks*, pages 1–9,

2019. URL <https://github.com/KatharaFramework/Kathara>. 2
- [6] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. 2
 - [7] Harrison Chase et al. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024. 2
 - [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>. 2
 - [9] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>. 2
 - [10] FRRouting Project. FRRouting: A free range routing protocol suite. <https://github.com/FRRouting/frr>, 2021. 2
 - [11] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. In *ACM SIGCOMM Computer Communication Review*, volume 44, pages 87–95, 2014. URL <https://arxiv.org/abs/1312.1719>. 2
 - [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>. 3